

Fine-Tuning Llama 3.1 for Feedback-to-Engineering Insights

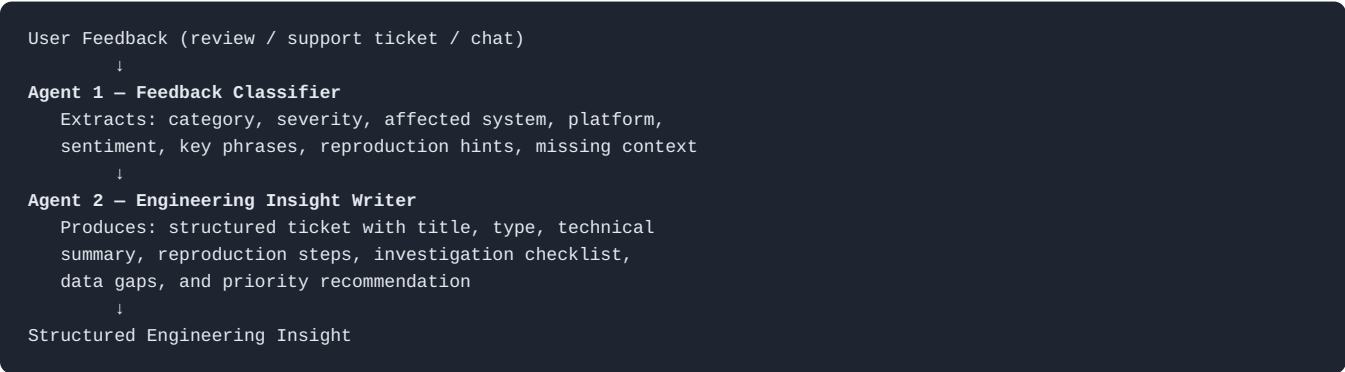
A step-by-step guide to install, configure, fine-tune, serve, and evaluate a two-agent pipeline that turns raw user feedback into structured engineering tickets — running entirely on your own machine.

Repository: github.com/praveenjatta/llama-finetuning-feedback-pipeline · Author: Praveen Kumar Jatta · jattaaai.com

1. What You're Building

This project fine-tunes **Llama 3.1 8B Instruct** locally to fix a specific failure mode: general-purpose models *hallucinate* on vague feedback. Handed "This app is garbage," a base model invents fake bug details, imaginary reproduction steps, and made-up system names. Engineers then waste time chasing problems that were never described.

The pipeline uses **two specialized agents**, both served from the same fine-tuned model:



After fine-tuning, vague inputs correctly produce an **INVESTIGATION_NEEDED** ticket with **SEVERITY: LOW** and a request for more detail — instead of fabricated specifics.

2. Prerequisites

Requirement	Detail / Why
Python 3.11	NOT 3.14 — PyTorch's MPS backend has a stack-overflow bug with 3.14's recursion handling on Apple Silicon.
Hardware	16 GB+ RAM. Tested on a 64 GB Apple Silicon MacBook. Training uses 4-bit quantization + LoRA to fit in memory.
Hugging Face account	With approved access to <code>meta-llama/Llama-3.1-8B-Instruct</code> (request access on the model page — approval is usually minutes).
HF access token	Create at huggingface.co → Settings → Access Tokens (read scope is enough).
OpenAI API key	Used only as the DeepEval judge model (GPT-4o) during evaluation — not for the pipeline itself.
ngrok account	Only needed if you run the model through the n8n cloud workflow. Not needed for the direct local evaluation path.
~35 GB free disk	Base model (~16 GB) plus two merged fine-tuned models (~16 GB total after sharding).

Note on model weights: The large model folders are excluded from the repo via `.gitignore`. You regenerate them locally by running the scripts below — nothing multi-gigabyte is downloaded from GitHub.

3. Installation

1 Clone the repository

```
git clone https://github.com/praveenjatta/llama-finetuning-feedback-pipeline.git
cd llama-finetuning-feedback-pipeline/code
```

2 Create a Python 3.11 virtual environment (recommended, keeps deps clean)

```
python3.11 -m venv .venv
source .venv/bin/activate # macOS / Linux
```

3 Install dependencies

```
pip3.11 install transformers peft trl datasets huggingface_hub \
bitsandbytes fastapi uvicorn torch deepeval openai \
python-dotenv requests
```

Apple Silicon tip: PyTorch will use the Metal (MPS) backend automatically. If `bitsandbytes` warns about CUDA, that's expected on Mac — 4-bit loading still works through the fallback path used by these scripts.

4. Configure Your Credentials

Two secrets are needed. **Never hard-code them into files you push to GitHub.**

Hugging Face token

The download, training, and merge scripts read `HF_TOKEN`. Set it in your shell before running them:

```
export HF_TOKEN="hf_your_token_here"
```

Important: The scripts currently contain a placeholder line `os.environ["HF_TOKEN"] = "YOUR_HF_KEY"`. Replace that placeholder with a read from the environment — e.g. `os.environ["HF_TOKEN"] = os.getenv("HF_TOKEN")` — or paste your token locally but do not commit it.

OpenAI key (for evaluation only)

```
export OPENAI_API_KEY="sk-proj-your_key_here"
```

5. Build the Model — Step by Step

1 Download the base model (~16 GB, roughly 10 minutes on a good connection)

```
python3.11 download_hf_model.py
```

Loads `meta-llama/Llama-3.1-8B-Instruct` with 4-bit NF4 quantization and caches it locally.

2 Fine-tune both agents with LoRA (~2.5 hours on Apple Silicon)

```
python3.11 fine_tune_training.py
```

This trains two separate LoRA adapters — one on `training_data_classifier.jsonl`, one on `training_data_insight.jsonl` (15 examples each). Key training settings baked into the script:

Setting	Value
LoRA rank (r) / alpha	16 / 32
Target modules	q_proj, k_proj, v_proj, o_proj
LoRA dropout	0.05
Max steps / epochs	100 / 1
Batch size × grad accumulation	2 × 4 (effective 8)
Learning rate / scheduler	2e-4 / linear
Max sequence length	2048

Outputs two adapter folders: `...-Classifier-v1-adapters` and `...-InsightWriter-v1-adapters`.

3 Merge adapters into standalone models

```
python3.11 merge_to_base_model.py
```

Merges each LoRA adapter back into the base weights and saves two self-contained models locally (sharded at 5 GB): `Llama-3.1-8B-Instruct-FineTuned-Classifier-v1` and `...-InsightWriter-v1`.

Heads-up: the push-to-Hub lines in this script are commented out and point at a placeholder namespace. Leave them commented — the merge saves everything you need locally. Only enable pushing if you first change the model IDs to your own Hugging Face account.

6. Serve the Model Locally

The pipeline exposes an OpenAI-compatible `/chat/completions` API so tools like n8n can call it.

Fine-tuned server (the one you'll use)

```
python3.11 load_run_local_finetune_model.py
```

Agent	Endpoint	Host / Port
Classifier	<code>/v2/chat/completions</code>	127.0.0.1 : 8006
Insight Writer	<code>/v3/chat/completions</code>	127.0.0.1 : 8006

Base server (optional — for before/after comparison)

```
python3.11 load_run_local_model.py
```

Serves the un-tuned base model at `/v1/chat/completions` on port **8005**. Useful for generating the "before" outputs yourself.

(Optional) Expose to n8n cloud via ngrok

```
ngrok http 8006
```

Then point your n8n LLM nodes at the public URL: Classifier → `{ngrok-url}/v2` , Insight Writer → `{ngrok-url}/v3` . Import `Fine_Tuning_n8n_-_using_local_LLM.json` as the workflow.

7. Run the Evaluation

Evaluation is fully automated with **DeepEval** — no manual copy-pasting of model outputs. The design separates output collection from scoring:

1. `run_full_eval.py` calls the fine-tuned model directly and collects all 8 test outputs for a phase.
2. Outputs are written to `outputs.properties`, keyed by phase (`before` / `after`).
3. `test_feedback_engineering.py` loads those outputs via `props_loader.py`, runs the metrics, and logs each run to the Confident AI dashboard.

Log in to DeepEval (once, to enable the dashboard)

```
deepeval login
```

Evaluate both phases

```
export OPENAI_API_KEY="sk-proj-your_key_here"
python3.11 test_feedback_engineering.py --phase before
python3.11 test_feedback_engineering.py --phase after
```

Each phase logs under a dashboard identifier — `llama-before-finetuning` and `llama-after-finetuning` — so the before/after comparison groups automatically in the Confident AI "Metrics breakdown" view.

The four metrics

Metric	What it checks
Engineering Groundedness	Does the output stick to what the user actually said (no invented systems)?
Engineering Usefulness	Is the ticket actionable for an engineer starting an investigation?
Answer Relevancy	Is the response relevant to the input feedback?
Vague Feedback Handling	Does the model correctly refuse to fabricate on vague input?

What good results look like

Metric (Vague set)	Before	After
Vague Feedback Handling (avg)	0.41	0.92
Vague Feedback Handling (pass rate)	25%	100%

Expected trade-off: Answer Relevancy on vague inputs drops ($\approx 0.95 \rightarrow 0.68$) because the tuned model now answers thin inputs with a thorough "insufficient data" ticket that the relevancy judge sees as less directly engaging. This is the correct engineering behavior and a known consequence of training heavily on vague-refusal examples.

8. File Reference

File	Role
<code>download_hf_model.py</code>	Downloads Llama 3.1 8B (4-bit)
<code>fine_tune_training.py</code>	LoRA fine-tunes both agents
<code>merge_to_base_model.py</code>	Merges adapters into standalone models
<code>load_run_local_finetune_model.py</code>	Serves fine-tuned models (ports /v2, /v3 on 8006)
<code>load_run_local_model.py</code>	Serves base model (/v1 on 8005)
<code>agent_prompts.py</code>	Shared system prompts for both agents
<code>run_full_eval.py</code>	Collects all 8 model outputs per phase
<code>props_loader.py</code>	Reads phase outputs from outputs.properties
<code>outputs.properties</code>	Stores 16 collected outputs (8 before + 8 after)
<code>test_feedback_engineering.py</code>	DeepEval suite, run by <code>--phase</code>
<code>test_plumbing.py</code>	Sanity check for local model plumbing
<code>training_data_classifier.jsonl</code>	15 training examples (Classifier)
<code>training_data_insight.jsonl</code>	15 training examples (Insight Writer)

9. Troubleshooting

Symptom	Fix
Stack overflow / recursion crash during training	You're on Python 3.14. Switch to 3.11.
<code>401 Incorrect API key</code> during eval	The literal placeholder was sent. Re-run <code>export OPENAI_API_KEY="sk-proj-..."</code> with your real key in the same terminal.
Gated repo / 403 on model download	Request access to the Llama 3.1 model on Hugging Face and confirm your <code>HF_TOKEN</code> is set.
Out of memory during fine-tune	Lower <code>per_device_train_batch_size</code> to 1 and raise <code>gradient_accumulation_steps</code> , or reduce <code>max_length</code> .
Dashboard comparison not grouping	Make sure both phases ran with <code>--phase</code> so each logs its identifier; confirm you ran <code>deepeval login</code> first.
n8n can't reach the model	Confirm the server is running on 8006 and the ngrok tunnel URL is current in the n8n LLM nodes.

Fine-Tuning Llama 3.1 for Feedback-to-Engineering Insights — Setup & Reproduction Guide.

Built by Praveen Kumar Jatta · Senior Technical Program Manager & AI Automation Consultant · jattaai.com

Fine-tuning is a last resort, not a first step — prompt engineering and evaluation come first. This project applies it to the one failure mode prompts couldn't fix: consistent refusal behavior on vague input.